



Chathurya V R 251801045-2025-AIDS ▾

**C2****Started on** Thursday, 17 September 2026, 8:23 AM**State** Finished**Completed on** Thursday, 17 September 2026, 8:23 AM**Time taken** 36 secs**Grade** 10.00 out of 10.00 (100%)

**Question 1** | Correct | Mark 10.00 out of 10.00**Playing with Chessboard:**

Ram is given with an  $n \times n$  chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position ( $n-1$ ,  $n-1$ ) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

**Example:****Input**

```
3
1 2 4
2 3 4
8 7 1
```

**Output:**

```
19
```

**Explanation:**

Totally there will be 6 paths among that the optimal is

Optimal path value:  $1+2+8+7+1=19$

**Input Format**

First Line contains the integer  $n$

The next  $n$  lines contain the  $n \times n$  chessboard values

**Output Format**

Print Maximum monetary value of the path

**Answer:** (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 #define MAX_N 500 // Adjust if n can be larger
4
5 int matrix[MAX_N][MAX_N];
6 int dp[MAX_N][MAX_N];
7
8 int main() {
9     int n;
10
11     // Read the grid size
12     if (scanf("%d", &n) != 1) return 0;
13
14     // Read the chessboard cell monetary values
15     for (int i = 0; i < n; i++) {
16         for (int j = 0; j < n; j++) {
17             if (scanf("%d", &matrix[i][j]) != 1) return 0;
18         }
19     }
20
21     // Initialize the starting position
22     dp[0][0] = matrix[0][0];
23
24     // Initialize the first row (can only come from the left cell)
25     for (int j = 1; j < n; j++) {
26         dp[0][j] = dp[0][j - 1] + matrix[0][j];
27     }
28
29     // Initialize the first column (can only come from the top cell)
30     for (int i = 1; i < n; i++) {
31         dp[i][0] = dp[i - 1][0] + matrix[i][0];
32     }
33
34     // Fill the rest of the DP table
35     for (int i = 1; i < n; i++) {
36         for (int j = 1; j < n; j++) {
37             int top = dp[i - 1][j];
38             int left = dp[i][j - 1];
39             dp[i][j] = max(top, left) + matrix[i][j];
40         }
41     }
42     printf("%d", dp[n-1][n-1]);
43 }
```

```

38         int left = dp[i][j - 1];
39
40         // Choose the path that gives the maximum monetary value
41         if (top > left) {
42             dp[i][j] = matrix[i][j] + top;
43         } else {
44             dp[i][j] = matrix[i][j] + left;
45         }
46     }
47 }
48
49 // The answer is at the bottom-right corner of the board
50 printf("%d\n", dp[n - 1][n - 1]);
51
52 return 0;

```

|   | Input                                         | Expected | Got |   |
|---|-----------------------------------------------|----------|-----|---|
| ✓ | 3<br>1 2 4<br>2 3 4<br>8 7 1                  | 19       | 19  | ✓ |
| ✓ | 3<br>1 3 1<br>1 5 1<br>4 2 1                  | 12       | 12  | ✓ |
| ✓ | 4<br>1 1 3 4<br>1 5 7 8<br>2 3 4 6<br>1 6 9 0 | 28       | 28  | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 10.00/10.00.

[Back to Course](#)